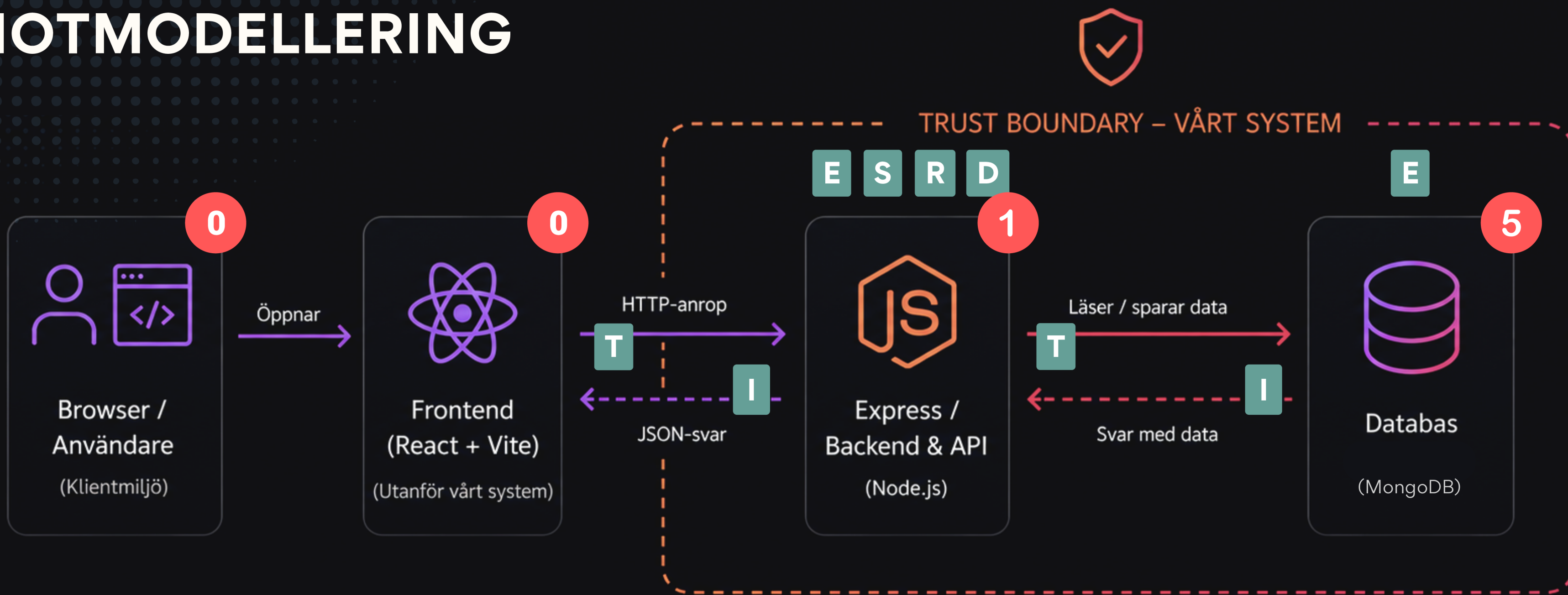


YH - MESSAGE APP

HOTMODELLERING



→ HTTP-request
(från klient till server)

- - - → JSON-response
(från server till klient)

→ Läser / sparar data
(server till databas)

- - - → Svar med data
(databas till server)

ABUSE CASE-FLÖDEN INOM SYSTEMKOMPONENTER

Hot och krav utifrån var i systemet de uppstår



BROWSER / KLIENTMILJÖ (UTANFÖR VÅRT SYSTEM)

Användarens browser och enhet ligger utanför systemets kontroll och betraktas därför inte som en trusted komponent i modellen.

Risker kopplade till klientmiljön hanteras genom säkerhetskrav i frontend och backend, exempelvis:

- ✓ Inputsanering och validering
- ✓ Autentisering och behörighetskontroll
- ✓ Säker tokenhantering
- ✓ Skydd mot manipulerade requests

KOMPONENT (från systemskiss)	FUNKTION/FLÖDE (exempel)	RISK/HOTSCENARIO (Vad kan gå fel?)	STRIDE-KATEGORI (Mest relevanta)	SÄKERHETSKRAV (Vad vi kräver av systemet)
FRONTEND (React+Vite)  Utanför vårt system	• Rendering av UI • Visa inloggningssida • Skriva meddelande • Redigera / radera meddelande • Läs meddelanden • Skicka HTTPS-anrop till API • Hantera svar och visa data	• Manipulerad klientlogik och XSS • Extremt långa inputs kan orsaka belastning eller krascher • NoSQL injections via osanerad input • Manipulerade requests via devtools • En användare försöker utge sig för att vara någon annan	• Tampering (T) • Tampering (T) & Denial of Service (D) • Tampering (T) • Tampering (T) • Spoofing (S) & Elevation of Privilege (E)	Användaren ska: • Endast kunna skriva in godkänd data • Inte kunna komma åt andra delar av applikationen via frontend • Inte kunna se känslig information via felmeddelanden • Logga in för att skapa, redigera och ta bort meddelanden
BACKEND+API (Express+Node.js) 	• Auth (login, logout) • Skapa / redigera / radera meddelande • Hämta meddelanden • Hantera användare • Verifiera tokens & behörigheter	• Broken access control (IDOR) • Mass assignment / manipulation av data • Dataläckage via felmeddelanden • Bruteforce / credential stuffing • Bristande autentisering och behörighetskontroll	• Elevation of Privilege (E) • Tampering (T) • Information Disclosure (I) • Denial of Service (D) • Spoofing (S) & Elevation of Privilege (E)	• Användare ska inte kunna skicka för många anrop från samma IP-adress • Användaren ska inte kunna försöka logga in för många gånger • Användaren ska bara kunna komma åt backend om den är behörig
DATABAS (MongoDB) 	• Lagra användare och lösenord • Lagra meddelanden • Lagra sessioner / tokens (vid behov)	• NoSQL injection och dataläckage • Klartextlagring av känslig data • Överexponerad data i API-svar • Datakorruption eller obehöriga ändringar	• Information Disclosure (I) • Tampering (T) • Information Disclosure (I) • Tampering (T)	• Lösenord lagras hashade i databasen • Databasen bör endast vara åtkomlig för backend • Databasen ska ha minst en säkerhetskopia

POTENTIELLA HOT

De största hoten som identifierats utifrån kritiska säkerhetsrisker inom varje systemkomponent

1 Frontend

Systemhaveri eller datamanipulation

Påverkan:
Systemet kraschar, betar sig oväntat eller visar manipulerad data.

Orsak:
Bristande inputvalidering och osanerad användardata

SR: 01 - 04

2 Frontend + Backend

Informationsläckage

Påverkan:
Känslig data exponeras genom felmeddelanden eller API-svar.

Orsak:
Systemet visar information som användaren inte ska se

SR: 05

3 Backend + API

Dataintrång eller obehörig åtkomst

Påverkan:
En angripare får åtkomst till funktioner, data eller användares information.

Orsak:
Bristande autentisering och behörighetskontroll

SR: 09

4 Databas

Kontoövertagande efter exponering

Påverkan:
Angripare kommer åt och missbruka användares inloggningsuppgifter.

Orsak:
Autentiseringsuppgifter lagras utan tillräckligt skydd

SR: 10

5 Databas

Dataläckage eller dataförlust

Påverkan:
Känslig data exponeras, manipuleras, raderas eller går förlorad.

Orsak:
Oskyddad lagring, bristande redudans och åtkomstkontroll

SR: 11



Dessa risker måste hanteras genom tydliga säkerhetskrav, kontroller och löpande tester för att säkerställa ett robust och säkert system.

SÄKERHETSKRAV



IDENTIFIERADE SÄKERHETSRISKER OCH HOTSCENARIER

Resultatet från hotmodelleringen utgör grunden för de krav som definieras i denna fas.

KRAVSPECIFIKATION

01

KRAV

Användaren ska endast kunna skriva in godkänd data på alla fält

Exempel: Validering och sanering av typ, längdbegränsning och filtrering av specialtecken

SR: 01-04

KATEGORI

Skadlig input

02

KRAV

Användaren ska inte kunna se känslig information via felmeddelanden

Exempel: Systemdetaljer och intern felinformation får inte exponeras

SR: 05

KATEGORI

**Informations-
läckage**

03

KRAV

Användaren ska kontrolleras för behörighet vid varje request

Exempel: Sessionskontroll, tokenvalidering och rollbaserad åtkomst

SR: 09

KATEGORI

Obehörig åtkomst

04

KRAV

Användarens lösenord bör lagras skyddade i databasen

Exempel: Lösenord ska aldrig lagras i klartext

SR: 10

KATEGORI

Dataläckage

05

KRAV

Användaren ska inte ha direkt klientåtkomst till databasen

Exempel: Databasen bör endast vara åtkomlig via backend

SR: 11

KATEGORI

**Dataläckage /
dataförlust**